

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 6 Exam Notes: Memory Management

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Contiguous Memory Allocation	2
1.1	Memory Partitioning Schemes	2
2	Non-Contiguous Memory Allocation	2
2.1	Paging Architecture	2
2.2	Segmentation	3
3	Virtual Memory and Demand Paging	3
3.1	Demand Paging & Page Faults	3
3.2	Page Replacement Algorithms	3
3.3	Thrashing	3

1. Contiguous Memory Allocation

Main memory must accommodate both the operating system and various user processes. In contiguous memory allocation, each process is contained in a single contiguous section of memory.

1.1 Memory Partitioning Schemes

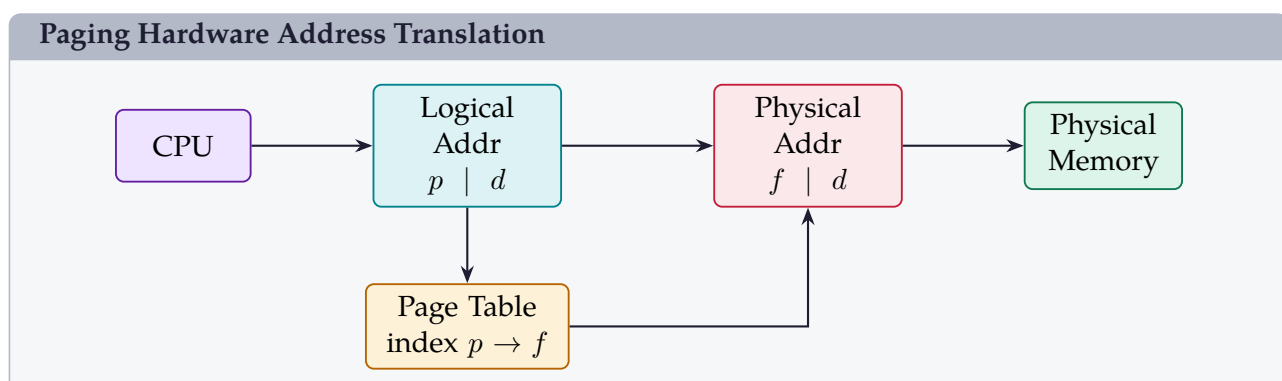
- **Fixed Partitioning (MFT - Multiprogramming with a Fixed number of Tasks):** Memory is divided into fixed-size partitions. Each partition contains exactly one process.
 - **Drawback:** Leads to **Internal Fragmentation** — if a process is smaller than the partition allocated to it, the remainder space is wasted.
- **Variable Partitioning (MVT - Multiprogramming with a Variable number of Tasks):** OS maintains a table of free memory slots (holes). When a process arrives, it is allocated a hole matching its size.
 - **Drawback:** Leads to **External Fragmentation** — total free memory is large enough to satisfy a request, but it is not contiguous (split into small holes).
 - **Solution: Compaction** — shifting all processes to one end of memory to group all free holes into a single block (possible only if dynamic relocation is used).

2. Non-Contiguous Memory Allocation

To eliminate external fragmentation and the need for compaction, modern systems use non-contiguous memory allocation.

2.1 Paging Architecture

- **Frames:** Physical memory is broken into fixed-size blocks (typically 4 KB).
- **Pages:** Logical memory is broken into blocks of the same size.
- **Address Translation:** Every address generated by the CPU is divided into:
 - **Page Number (p):** Used as an index into a **Page Table** to find the base physical frame.
 - **Page Offset (d):** Combined with the base frame address to yield the physical address.



- **TLB (Translation Lookaside Buffer):** A fast hardware cache of associative registers used to store recent page-to-frame translations. It reduces the memory access penalty from two cycles (one for page table, one for data) to close to one cycle.

2.2 Segmentation

Segmentation is a memory-management scheme that supports the programmer's view of memory. A logical address space is a collection of variable-length segments (e.g., main program, procedure, stack, variables).

- **Segment Table:** Maps logical address $\langle \text{segment_number}, \text{offset} \rangle$ to physical addresses. Each entry has a **base** (start physical address) and **limit** (length of segment).

3. Virtual Memory and Demand Paging

Virtual Memory allows execution of processes that are not completely in main memory. It enables extremely large logical address spaces.

3.1 Demand Paging & Page Faults

Pages are loaded into RAM only when they are referenced during execution (lazy swapper).

- **Page Table Entry:** Includes a **valid-invalid bit** (V/I). V means page is in memory; I means page is on disk.
- **Page Fault:** Occurs when a running process references a page marked as invalid.
- **Page Fault Handling Sequence:**

1. Trap to OS on page fault. 2. Save process registers and state. 3. Verify reference is valid, find page location on disk. 4. Read page from disk into a free frame (using a page replacement algorithm if no free frames exist). 5. Update page table (set bit to V). 6. Restart the instruction that was interrupted by the trap.

3.2 Page Replacement Algorithms

When no frames are free, the OS must swap a page out to make room.

- **FIFO (First-In, First-Out):** Replaces the oldest page. Prone to **Belady's Anomaly** — page fault rate increases as physical frame allocation increases.
- **Optimal (OPT):** Replaces the page that will not be used for the longest time in the future. Optimal benchmark, but impossible to implement (requires future knowledge).
- **LRU (Least Recently Used):** Replaces the page that has not been referenced for the longest time in the past. Requires hardware support (counter or stack).

3.3 Thrashing

Thrashing is a state where a process spends more time paging (swapping pages in and out of disk) than executing useful instructions.

- **Cause:** The process has been allocated fewer frames than its **working set** (the set of pages it is actively using). As a result, it page faults immediately, forcing page swaps in a continuous loop.
- **Solution:** Working-Set Model (allocates frames based on active page locality) or Page-Fault Frequency (PFF) control.

Quick Revision — Module 6

Memory Management | OE-EC604B

Virtual Memory Math

$$\text{Effective Access Time (EAT)} = (1 - p) \times T_{\text{mem}} + p \times T_{\text{fault}}$$

where p = Page Fault Rate ($0 \leq p \leq 1$), T_{mem} = RAM Access Time, T_{fault} = Page Fault Service Time (disk seek + read).

One-Line Definitions

- **Internal Fragmentation:** Wasted memory inside a fixed partition allocated to a smaller process.
- **External Fragmentation:** Free memory blocks distributed non-contiguously, preventing allocation.
- **Paging:** Non-contiguous allocation breaking physical memory into fixed frames and logical into pages.
- **TLB:** Translation Lookaside Buffer; hardware cache to speed up logical-to-physical address mapping.
- **Segmentation:** Memory allocation method based on logical units (code, stack, global data).
- **Demand Paging:** Lazy page allocation technique loading pages only on execution references.
- **Belady's Anomaly:** Phenomenon where allocating more physical frames increases page faults in FIFO.
- **Thrashing:** System state where CPUs sit idle because processes are locked in page-swap loops.

Frequently Asked / Viva Questions

Q1. Compare Paging and Segmentation.

Paging divides memory into fixed-size blocks (pages/frames), avoiding external fragmentation but suffering from internal fragmentation. It is invisible to the user. Segmentation divides memory into logical variable-size modules, matching the programmer's view, but suffers from external fragmentation.

Q2. Given a reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3. Frame size = 3.

Calculate page faults for: (a) FIFO, (b) LRU, (c) Optimal.

(a) FIFO: Frames status:

- Ref 7: [7] - Fault 1

Frequently Asked / Viva Questions (contd.)

- **Ref 0:** [7, 0] - Fault 2
- **Ref 1:** [7, 0, 1] - Fault 3
- **Ref 2:** [2, 0, 1] (replaces 7) - Fault 4
- **Ref 0:** [2, 0, 1] - Hit
- **Ref 3:** [2, 3, 1] (replaces 0) - Fault 5
- **Ref 0:** [2, 3, 0] (replaces 1) - Fault 6
- **Ref 4:** [4, 3, 0] (replaces 2) - Fault 7
- **Ref 2:** [4, 2, 0] (replaces 3) - Fault 8
- **Ref 3:** [4, 2, 3] (replaces 0) - Fault 9. **Total Faults = 9.**

(b) **LRU:** Frames status:

- **Ref 7, 0, 1:** [7, 0, 1] - Faults = 3
- **Ref 2:** [2, 0, 1] (replaces 7) - Fault 4
- **Ref 0:** [2, 0, 1] - Hit (0 is most recent)
- **Ref 3:** [2, 0, 3] (replaces 1) - Fault 5
- **Ref 0:** [2, 0, 3] - Hit
- **Ref 4:** [4, 0, 3] (replaces 2) - Fault 6
- **Ref 2:** [4, 0, 2] (replaces 3) - Fault 7
- **Ref 3:** [3, 0, 2] (replaces 4) - Fault 8. **Total Faults = 8.**

(c) **Optimal:** Frames status:

- **Ref 7, 0, 1:** [7, 0, 1] - Faults = 3
- **Ref 2:** [2, 0, 1] (replaces 7, unused furthest) - Fault 4
- **Ref 0:** [2, 0, 1] - Hit
- **Ref 3:** [2, 0, 3] (replaces 1, unused furthest) - Fault 5
- **Ref 0:** [2, 0, 3] - Hit
- **Ref 4:** [4, 0, 3] (replaces 2, unused furthest) - Fault 6
- **Ref 2:** [2, 0, 3] (replaces 4) - Fault 7
- **Ref 3:** [2, 0, 3] - Hit. **Total Faults = 7.**

Q3. What is Belady's Anomaly? Give an example of when it occurs.

Belady's Anomaly is when allocating more page frames to a process increases the number of page faults. It occurs in FIFO page replacement. Example string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Frame size = 3 gives 9 faults; Frame size = 4 gives 10 faults.

Q4. What is a TLB and how does it improve Effective Access Time?

A TLB is a fast associative cache on the CPU containing translation table entries. Instead of doing two memory accesses (one for page table in RAM, one for target location), the CPU checks the TLB. If it's a hit, the translation is resolved in < 1 cycle, minimizing EAT.

Frequently Asked / Viva Questions (contd.)

Q5. Explain the concept of compaction in memory management.

Compaction is a technique used to resolve external fragmentation in variable partitioning. The OS dynamically shifts active processes in memory to make them contiguous, merging all small free holes into a single large free partition.

Q6. What is thrashing? How can it be detected and prevented?

Thrashing occurs when a process is constantly swapping pages in and out of disk because it has been allocated fewer frames than its active working set. Detected when CPU utilization drops and disk queuing spikes. Prevented using the Working-Set Model or Page-Fault Frequency controls.

Q7. Describe the page fault handling mechanism.

Process references page → MMU detects invalid bit → traps to OS kernel → OS checks virtual address and finds page on disk → allocates free frame → schedules I/O to read page → updates page table to valid → resumes the process.

Q8. How does logical address space differ from physical address space?

Logical address space is the set of all virtual addresses generated by a program during compilation/execution. Physical address space is the set of actual physical hardware addresses in main memory (RAM).

Q9. Explain the difference between internal and external fragmentation.

Internal fragmentation occurs when allocated memory is slightly larger than the process's demand, leaving wasted space inside the partition. External fragmentation occurs when free memory is sufficient but broken into small, non-contiguous blocks.

Q10. What is a dirty bit (modify bit) and how does it optimize page replacement?

A dirty bit is a flag set in the page table entry when a page is written to. During page replacement, if the dirty bit is 0 (page unchanged), the OS can overwrite the frame without writing it back to disk, halving page-fault overhead.

Page Replacement Algorithms summary

Algorithm	Implementation Complexity	Hardware Support?	Belady's Anomaly?
FIFO	Very Low	None	Yes
Optimal	Impossible (needs future data)	None	No
LRU	High (requires stacks/counters)	Yes (clock/reference bits)	No